1. Feature-by-Feature Comparison Table

Feature / Aspect	8085 Microprocessor	8086 Microprocessor	Simple Memory Trick / Analogy
Data Bus Width	**8 bits** (transfers 1 byte)	**16 bits** (transfers 2 bytes at once)	1-lane road vs. 2-lane highway
Address Bus Width	**16 bits**	**20 bits**	Address size: 16 digits vs. 20 digits
Memory Capacity	**64 KB** (2^{16} bytes)	**1 MB** (2^{20} bytes)	Small room vs. large warehouse
Internal Architecture	Single unified processing block	Split into 2 units: **BIU** and **EU**	Solo worker vs 2-person team
Pipelining /Pre-fetching	**No** (wait-and-fetch)	**Yes** (6-byte Instruction Queue)	One dish at a time vs. assembly line
Memory Management	Single flat memory	**Segmented Memory** (CS, DS, SS, ES)	One big drawer vs. organized dividers
General Registers	8-bit registers: B, C, D, E, H, L	16-bit registers: AX, BX, CX, DX (split into AH, AL, etc.)	Half-size storage lockers
Math Capabilities	Basic (+, -). No hardware multiply/divide	Built-in hardware MUL and DIV	Basic calculator vs. scientific
Operating Modes	Single mode	**Minimum** (single CPU) & **Maximum** (multi-CPU)	Standalone vs. Team Mode
Hardware Interrupts	5 direct pins (TRAP, RST 7.5, etc.)	256 vector types (INTR, NMI)	5 panic buttons vs. 256 switchboard lines

Serial I/O Pins	Dedicated SID / SOD pins	No direct SID/SOD pins (uses peripheral ICs)	Direct serial port on chip

2. Working Principle & Workflow Differences

The **working principle** determines how the processor gets work done from start to finish.

8085 Workflow (Sequential / Non-Pipelined):
[1. Fetch] ➔ [2. Decode] ➔ [3. Execute] ➔ [4. Fetch Next] ➔ [5. Decode Next] ...
(The processor sits idle while waiting for each step to finish.)

8086 Workflow (Pipelined / Parallel Processing):
BIU Unit: [Fetch Inst 1] ➔ [Fetch Inst 2] ➔ [Fetch Inst 3] ➔ [Fetch Inst 4] ...
EU Unit: [Decode & Exec 1] ➔ [Decode & Exec 2] ➔ [Decode & Exec 3] ...
(BIU pre-fills a 6-byte queue so EU never waits.)

8085 Working Principle: "Sequential Single-Tasker"

1. **Fetch:** Program Counter (PC) sends a 16-bit address via A8–A15 and multiplexed AD0–AD7. Memory sends the instruction byte back.
2. **Decode:** The 8-bit instruction is loaded into the Instruction Register (IR) and translated by the control unit.
3. **Execute:** The ALU runs the operation using the Accumulator (A). **The entire processor halts fetching new data until this execution completes.**
4. **Update:** PC increments, and the next cycle begins from scratch.

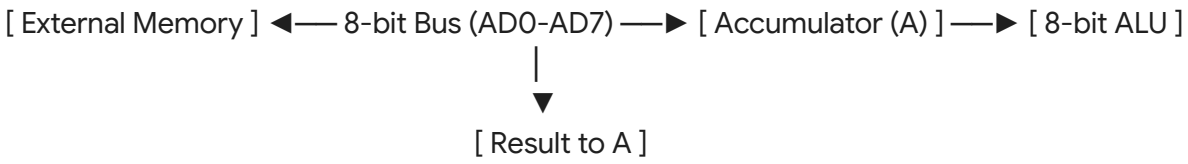
8086 Working Principle: "Assembly Line (Pipelined)"

1. **Parallel Execution:** The job is split between two internal units operating simultaneously:
   - **Bus Interface Unit (BIU):** Acts like a supplier. It calculates physical memory addresses and pre-fetches up to **6 bytes of future instructions** into an internal **instruction queue.**
   - **Execution Unit (EU):** Acts like a worker. It pulls instructions straight out of the queue, decodes them, and runs them in the 16-bit ALU.
2. **Zero Idle Time:** While the EU is busy executing a complex math operation, the BIU is already fetching the next instructions from memory over the 16-bit bus.

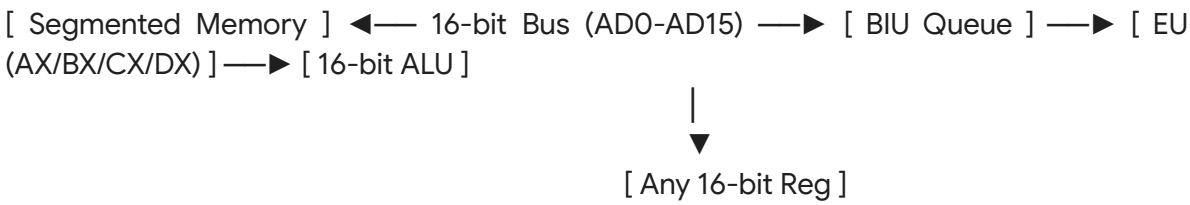
3. Dataflow Differences (How Data Moves Inside)

Dataflow refers to the actual path data bytes take between buses, registers, and memory during execution.

8085 Dataflow Path:



8086 Dataflow Path:



8085 Dataflow Characteristics

- **8-bit Bottleneck:** The external data bus (AD0–AD7) carries only 1 byte at a time. To transfer a 16-bit number, the 8085 must perform **two back-to-back memory trips**.
- **Accumulator-Centric Traffic:** Almost all calculations require data to flow through the **Accumulator (A)**. The ALU takes one byte from A, one byte from a register/memory, and stores the answer back in A.
- **Flat Address Calculation:** Address generation is direct—the 16-bit Program Counter sends its address directly to the bus pins.

8086 Dataflow Characteristics

- **16-bit Parallel Transfer:** The data bus (AD0–AD15) carries 2 bytes simultaneously in a single clock cycle.
- **Flexible Register Paths:** Data can flow freely between any general-purpose registers (AX, BX, CX, DX) and the ALU without being forced through a single accumulator.
- **Segmented Address Calculation:** Data addresses undergo a calculation step inside the BIU before hitting the pins:
- $PhysicalAddress(20-bit) = (SegmentRegister \times 16) + OffsetRegister$

  This allows data streams to be neatly separated into **Code**, **Data**, **Stack**, and **Extra.** segments.